

AIM - Data Science & AI Business Project



Smart Lead Management

GR01: TzuChing Lee, *Anjali Barvaliya*, Adarsh Raj

Agenda of Final Presentation

- Introduction
- Business Understanding
- Data Understanding
- Modeling & Evaluation
 - XGBoost
 - Decision Tree
 - Logistisc Regression
- Conclusion

Core Team Members



Adarsh Raj
Data Scientist



Anjali
Data Scientist



TzuChing Lee
Data Scientist

Stakeholder



Monika Frey-Luxemburger
Owner of X Education

Overview CRIPS DM – Phases, Tasks and Milestones



Timeline					
04/10 – 04/23/2025	04/24 – 05/07/2025	05/08 – 05/21/2025	05/22 – 06/18/2025	06/19 – 07/09/2025	07/10 – 07/17/2025
• Determine Business Objectives • Assess Situation • Determine Data Mining Goals • Produce Project Plan	• Collect Initial Data • Describe Data • Explore Data • Verify Data Quality	• Select Data • Clean Data • Construct Data • Integrate Data • Format Data Dataset Description	• Select Modelling Techniques • Generate Test Design • Build Model • Assess Model	• Evaluate Results • Review Process • Determine Next Steps	• Plan Deployment • Plan Monitoring and Maintenance • Produce Final Report • Review Project

Milestones

06/05/2025
Milestone
Presentation

06/26/2025
Final
Presentation

07/17/2025
Submit PDF

Name	Responsibility
Adarsh Raj	Introduction, Business Understanding & Data Understanding Model - XGBoost
Anjali	Model - Decision Tree
TzuChing Lee	Model – Logistic Regression, Conclusion

Business Objective:

X Education wants to **improve course sales** by finding which **leads** are most likely to **convert**.

Challenge:

Only about **30% of leads** convert. Treating all leads the same wastes time and effort. The data has **missing values** and is **unbalanced**.

Goal:

Build a **lead scoring model** that assigns conversion probabilities using machine learning.

Target: Achieve **~80% precision**, enabling the sales team to prioritize effectively.

Data Understanding - Collect Initial Data

- The dataset contains historical lead data from X Education, including profile and behavioral features such as lead source, website activity, and referral status. Each lead is labeled as converted or not.

Characteristic	Description
 Total Records	9240
 Total Features	37
 Duplicate Rows	0
 Missing Values	Significant in some columns
 Numerical Features	TotalVisits, Time Spent, Converted
 Categorical Features	Lead Origin, Source, Country
 Binary Flags	Do Not Email, Receive Updates
 Target Variable	Converted
 Conversion Rate	Approximately 39%
 Marketing Channels	Search, Digital Advertisement
 Engagement & Quality	Asymmetrique Scores, Lead Quality
 Identifiers & Metadata	Prospect ID, Lead Number

Data Understanding - Data Description

Intial Dataframe – 9240 * 37

[163]: df.shape

[163]: (9240, 37)

[157]: print("\nFirst 5 rows:\n")
df.head(5)

First 5 rows:

[157]:

	Prospect ID	Lead Number	Lead Origin	Lead Source	Do Not Email	Do Not Call	Converted	TotalVisits	Total Time Spent on Website	Page Views Per Visit	...	Get updates on DM Content	Lead Profile	City	Asymmetrique Activity Index	Asymmetrique Profile Index	Asymmetrique Activity Score	Asymmetrique Profile Score	I agree to pay the amount through cheque	A free copy of Mastering The Interview	Last Notable Activity
0	7927b2df-8bba-4d29-b9a2-b6e0beafe620	660737	API	Olark Chat	No	No	0	0.0	0	0.0	...	No	Select	Select	02.Medium	02.Medium	15.0	15.0	No	No	Modified
1	2a272436-5132-4136-86fa-dcc88c88f482	660728	API	Organic Search	No	No	0	5.0	674	2.5	...	No	Select	Select	02.Medium	02.Medium	15.0	15.0	No	No	Email Opened
2	8cc8c611-a219-4f35-ad23-fdfd2656bd8a	660727	Landing Page Submission	Direct Traffic	No	No	1	2.0	1532	2.0	...	No	Potential Lead	Mumbai	02.Medium	01.High	14.0	20.0	No	Yes	Email Opened
3	0cc2df48-7cf4-4e39-9de9-19797f9b38cc	660719	Landing Page Submission	Direct Traffic	No	No	0	1.0	305	1.0	...	No	Select	Mumbai	02.Medium	01.High	13.0	17.0	No	No	Modified
4	3256f628-e534-4826-9d63-4a8b88782852	660681	Landing Page Submission	Google	No	No	1	2.0	1428	1.0	...	No	Select	Mumbai	02.Medium	01.High	15.0	18.0	No	No	Modified

5 rows x 37 columns

Final Dataframe – 9240 * 75

4: Final Processed Data - df_final

[102]: #Processed Dataframe - df_final
df_final.shape

[102]: (9240, 75)

[161]: df_final.head(5)

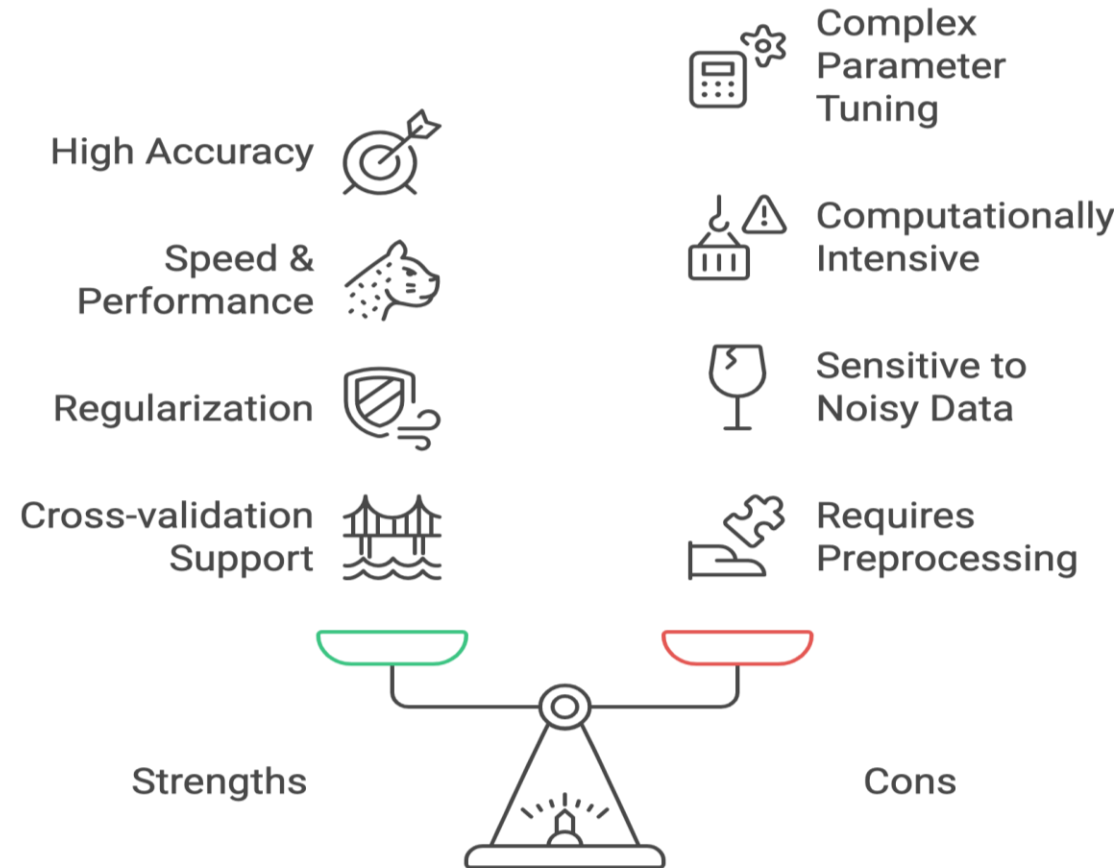
[161]:

	Lead Origin_API	Lead Origin_Landing Page Submission	Lead Origin_Lead Add Form	Lead Origin_Lead Import	Lead Origin_Quick Add Form	Lead Source_Direct Traffic	Lead Source_Facebook	Lead Source_Google	Lead Source_Olark Chat	Lead Source_Organic Search	...	Last Notable Activity_Unreachable	TotalVisits	Page Views Per Visit	Total Time Spent on Website	amount_missing	Asymmetrique Activity Index	Asymmetrique Profile Index	Asymmetrique Activity Score	Asymmetrique Profile Score	Converted
0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	...	0.0	-1.155592	-1.272994	-0.906301	-0.826392	-0.24773	-0.925147	0.595387	-0.756949	-0.791863
1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.650619	0.129372	0.368304	-1.112733	-0.24773	-0.925147	0.595387	-0.756949	-0.791863
2	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	...	0.0	-0.433108	-0.151101	1.990873	-1.112733	-0.24773	1.418538	-0.257090	2.565777	1.262845
3	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	...	0.0	-0.794350	-0.712048	-0.329514	-1.112733	-0.24773	1.418538	-1.109568	0.572142	-0.791863
4	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	...	0.0	-0.433108	-0.712048	1.794198	-1.112733	-0.24773	1.418538	0.595387	1.236687	1.262845

5 rows x 75 columns

What is XGBoost?

- **XGBoost (Extreme Gradient Boosting)** is a **supervised learning** algorithm.
- It is based on **ensemble learning** and specifically uses **gradient boosting** over decision trees to optimize predictive performance.
- We used it for a **binary classification** problem (Converted as the target, **Imbalanced** data - (38.5% as Class 1 61.5% as Class 0).



Specific Challenges in Model Implementation

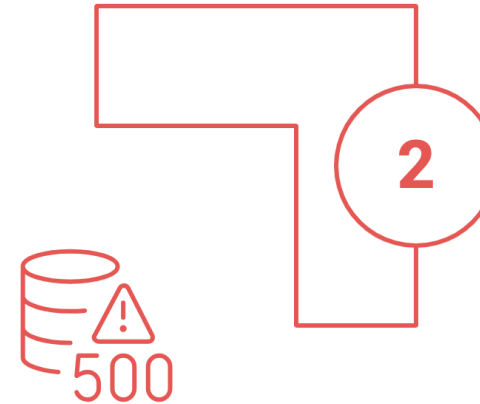
Interpreting SHAP Values

Interpreting SHAP values is simple but crucial for understanding model insights.



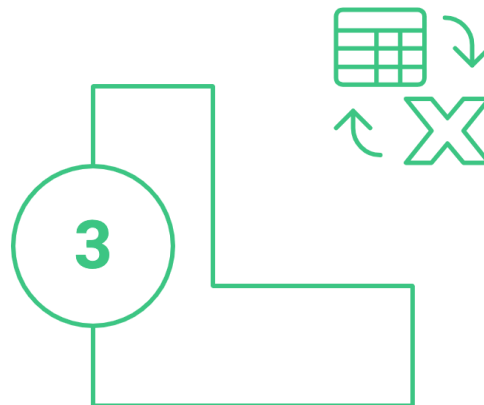
Overfitting Risk

Overfitting risk is complex and significantly impacts model performance.



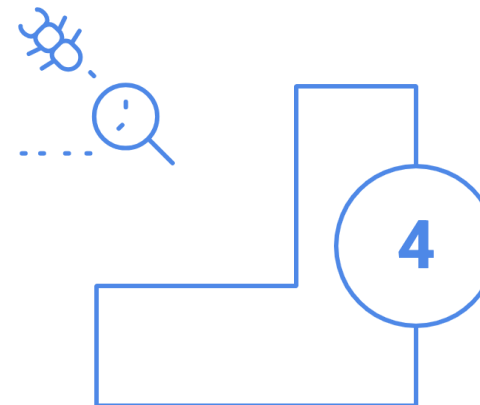
Handling Sparse Matrix Conversion

Handling sparse matrix conversion is straightforward with minimal impact.



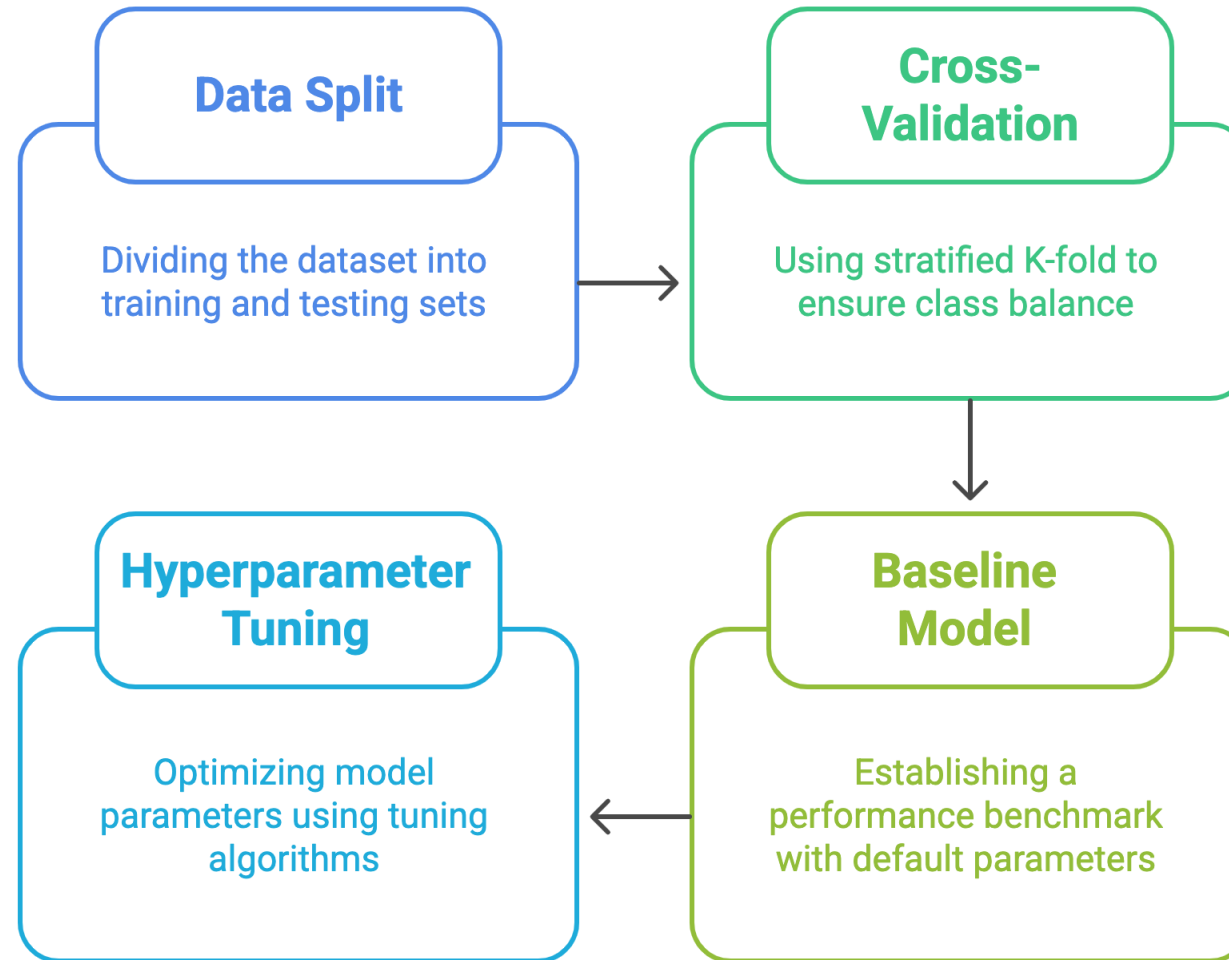
Hyperparameter Tuning

Hyperparameter tuning is complex but has limited impact on model outcomes.



Hyperparameter	Value	Explanation
 colsample_bytree	0.5398	Uses ~54% features per tree
 gamma	1.5273	Larger gain needed for split
 learning_rate	0.1092	Slower learning rate
 max_depth	6	Moderately deep trees
 min_child_weight	4	More conservative split
 n_estimators	323	Number of boosting rounds
 reg_alpha	0.3829	L1 regularization applied
 reg_lambda	0.2249	L2 regularization applied
 scale_pos_weight	0.7866	Adjusts for positive class imbalance
 subsample	0.7915	Uses ~79% training samples

Model Testing Strategy



Base XGB Model

Split Ratio	Precision	Recall	F1 Score	ROC-AUC
80/20	0.9415	0.9270	0.9342	0.9841
70/30	0.9349	0.9139	0.9242	0.9837
90/10	0.9425	0.9213	0.9318	0.9809

Optimised XGB Model

Split Ratio	Precision	Recall	F1 Score	ROC-AUC
80/20	0.9504	0.9143	0.9320	0.9422
70/30	0.9456	0.9110	0.9280	0.9391
90/10	0.9444	0.9073	0.9255	0.9369

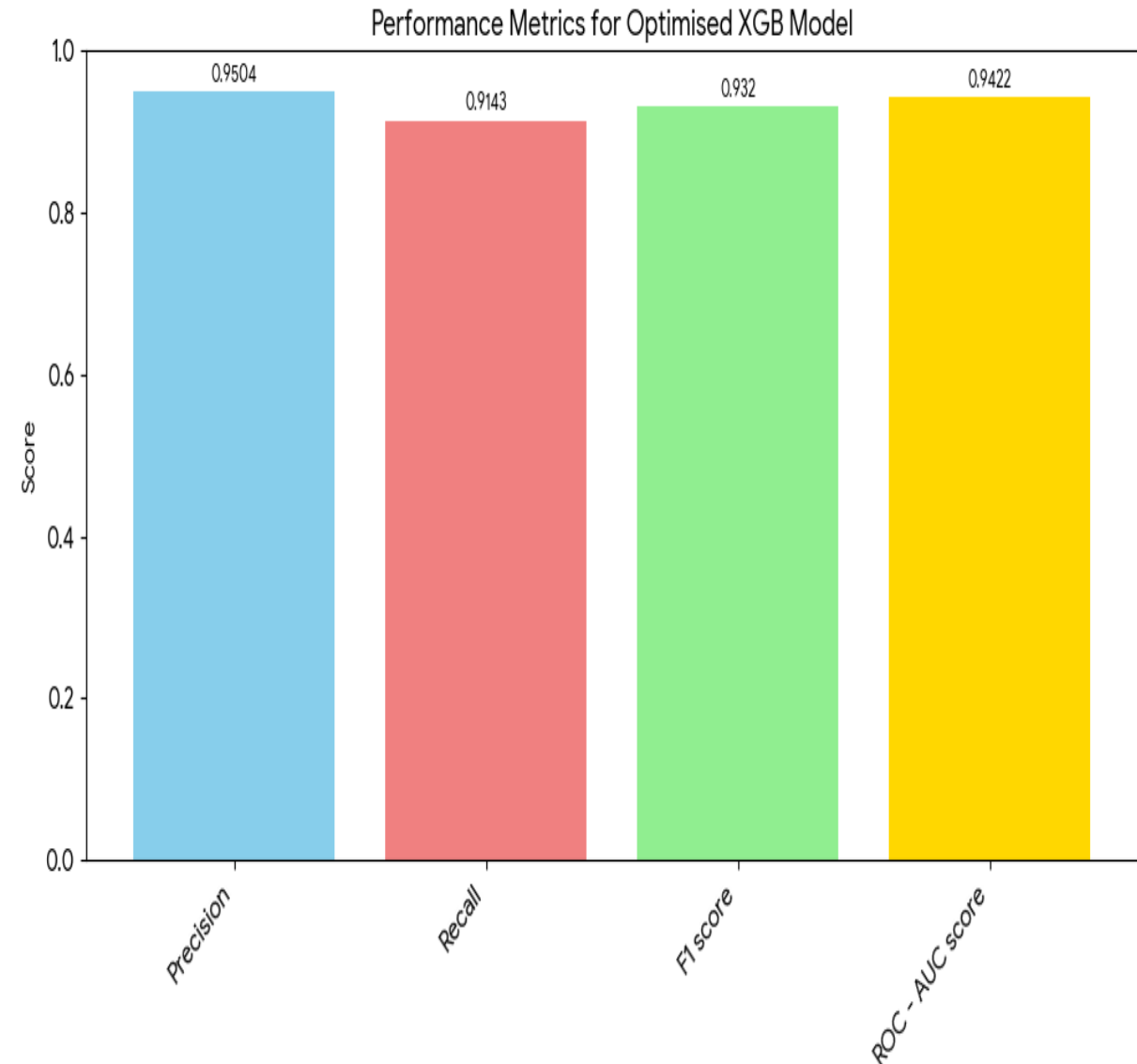
XGBoost – Optimisized Performance Metrics(80/20 split)

```
X_test_pp=X_test.copy()
xgb_pred_test = xgb_randomcv.predict(X_test_pp)
y_proba = xgb_randomcv.predict_proba(X_test_pp)[: , 1]
print('Optimised Model : XGB')
print("Precision:", precision_score(y_test,xgb_pred_test))
print("Recall:", recall_score(y_test,xgb_pred_test))
print("F1 score:", f1_score(y_test,xgb_pred_test))
print("ROC - AUC score:", roc_auc_score(y_test,xgb_pred_test))
```

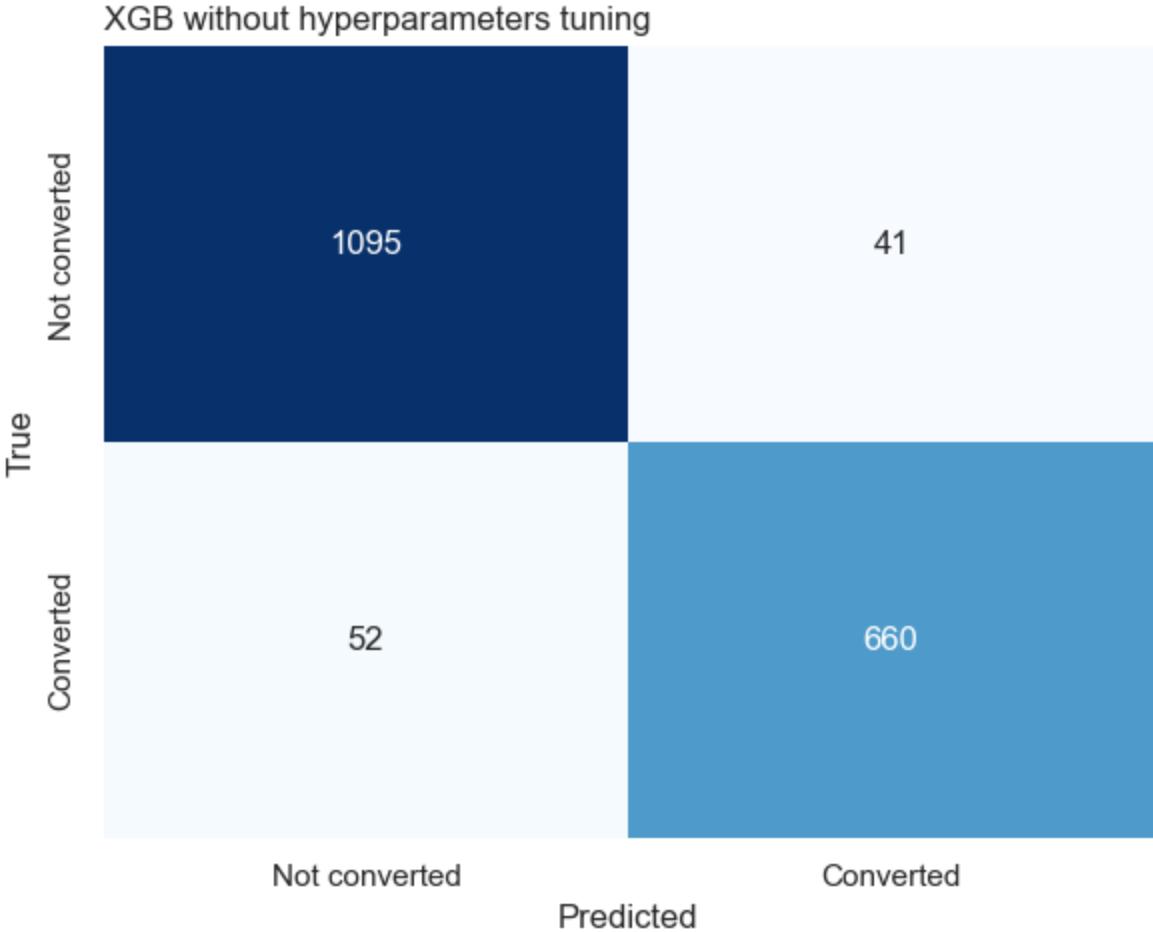
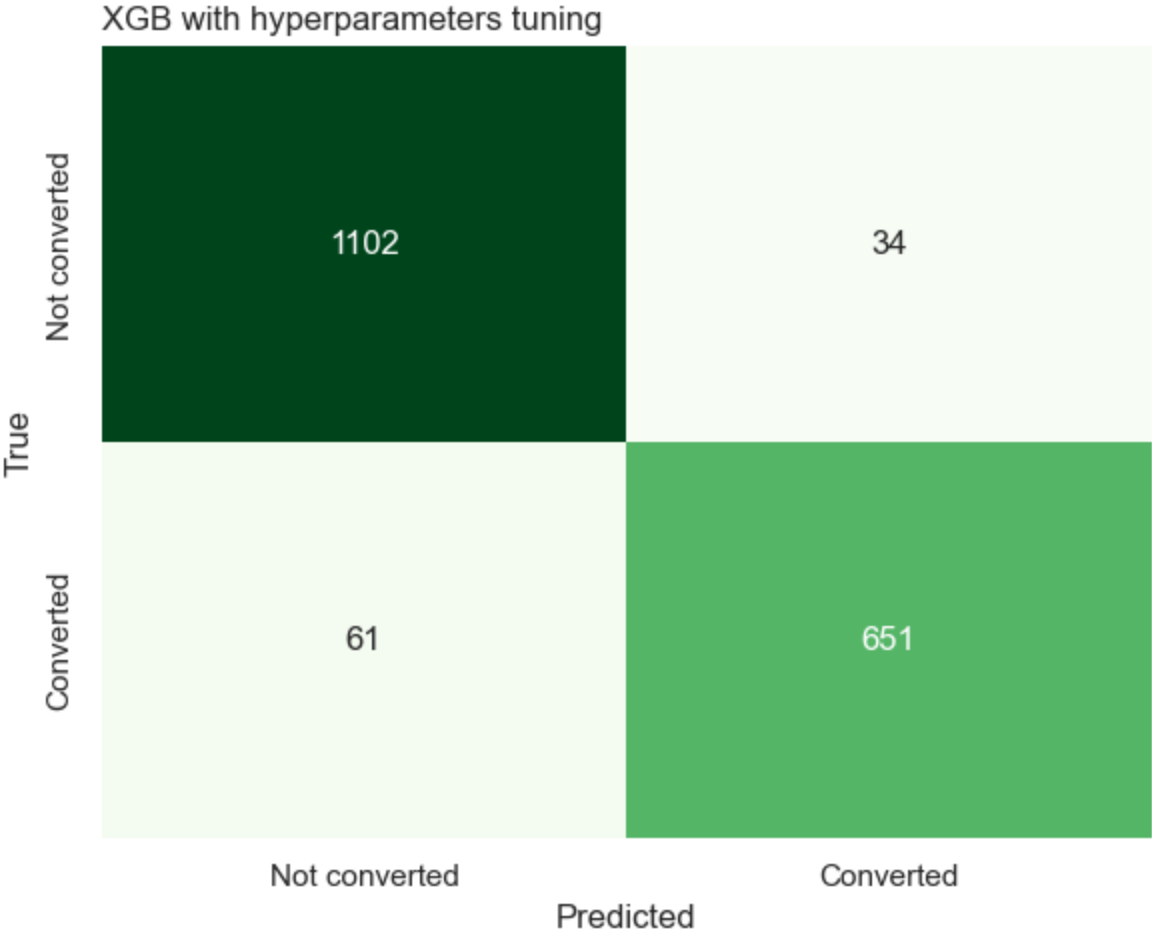
Optimised Model : XGB
Precision: 0.9503649635036496
Recall: 0.9143258426966292
F1 score: 0.9319971367215462
ROC - AUC score: 0.9421981326159202

Confusion Matrix (rows = Actual, columns = Predicted):

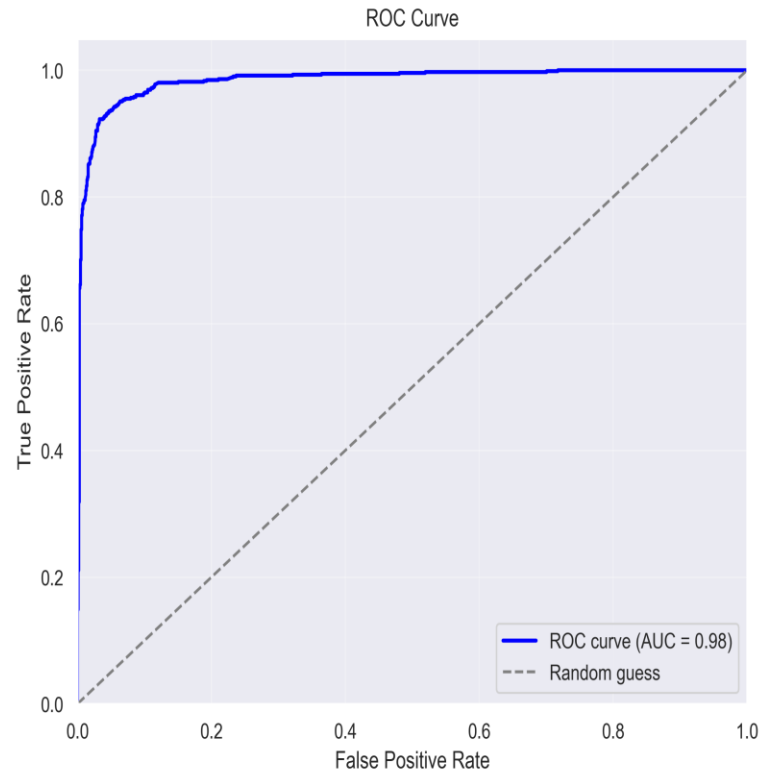
	Predicted: 0	Predicted: 1
Actual: 0	1102	34
Actual: 1	61	651



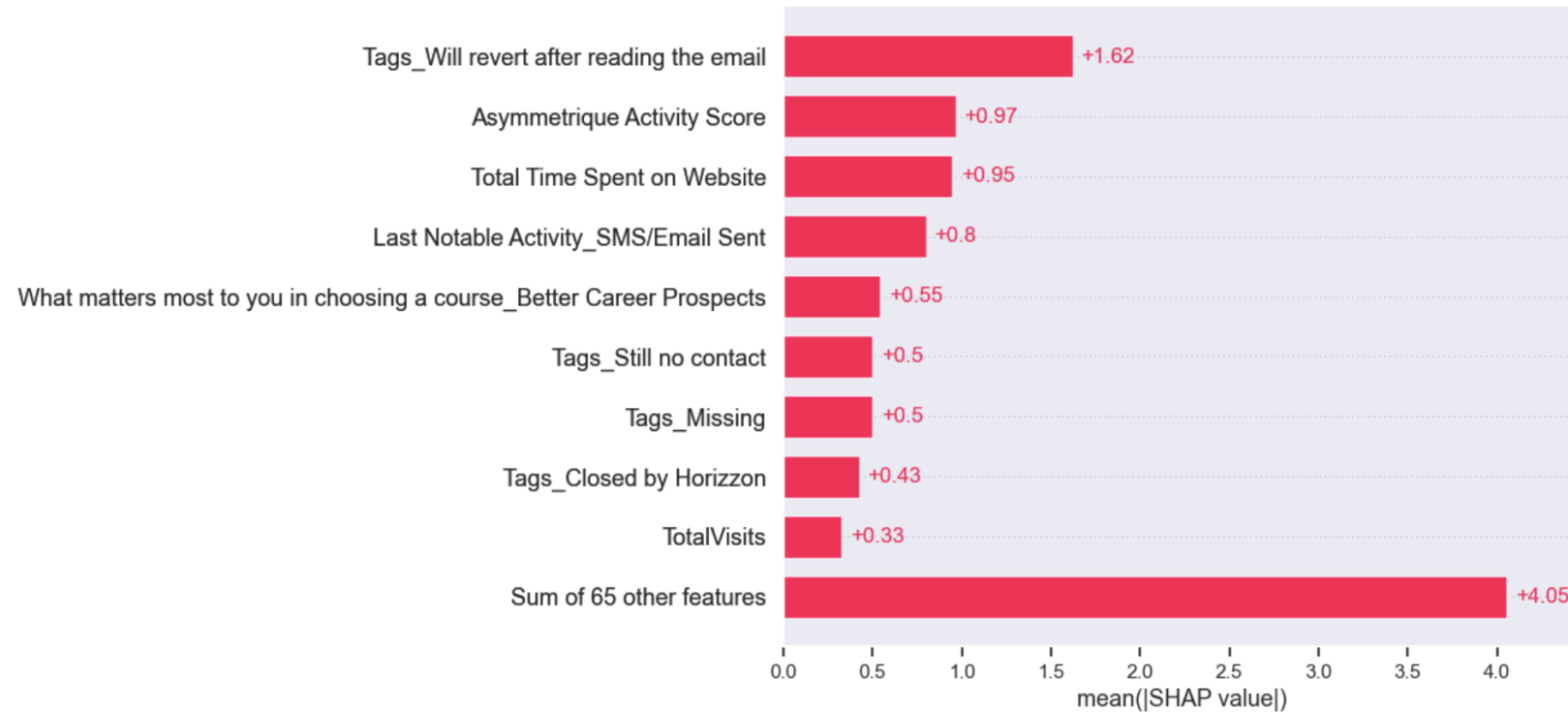
XGBoost – Confusion Matrix Comparison

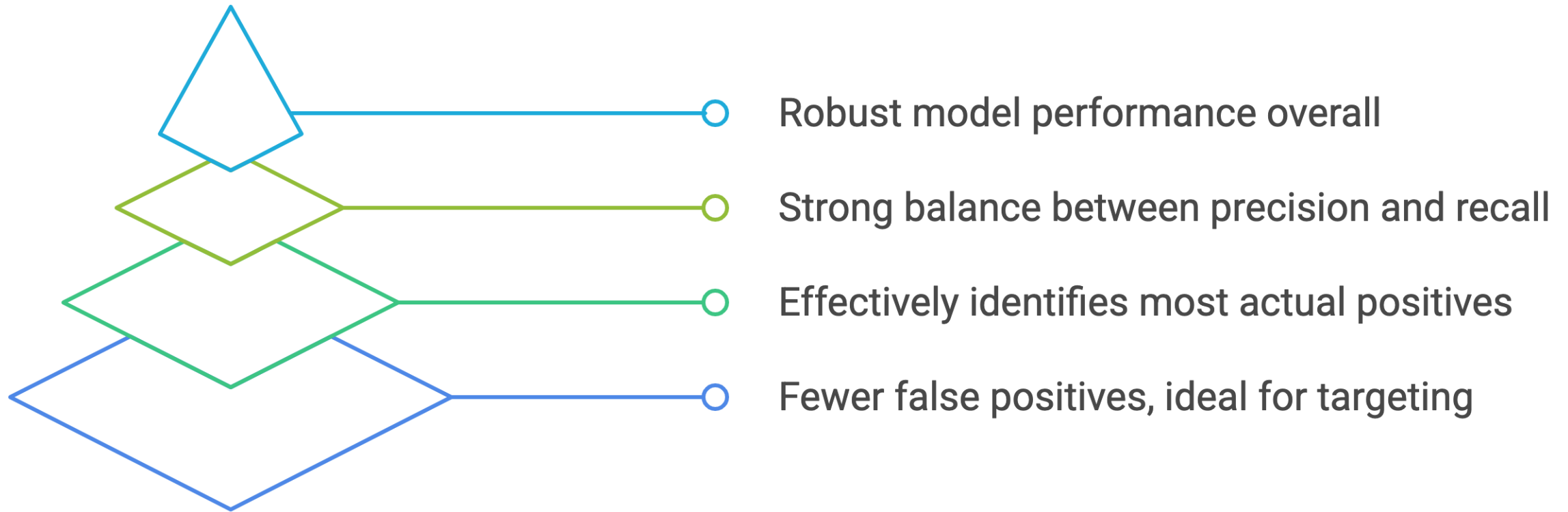


ROC Curve



Feature Importance

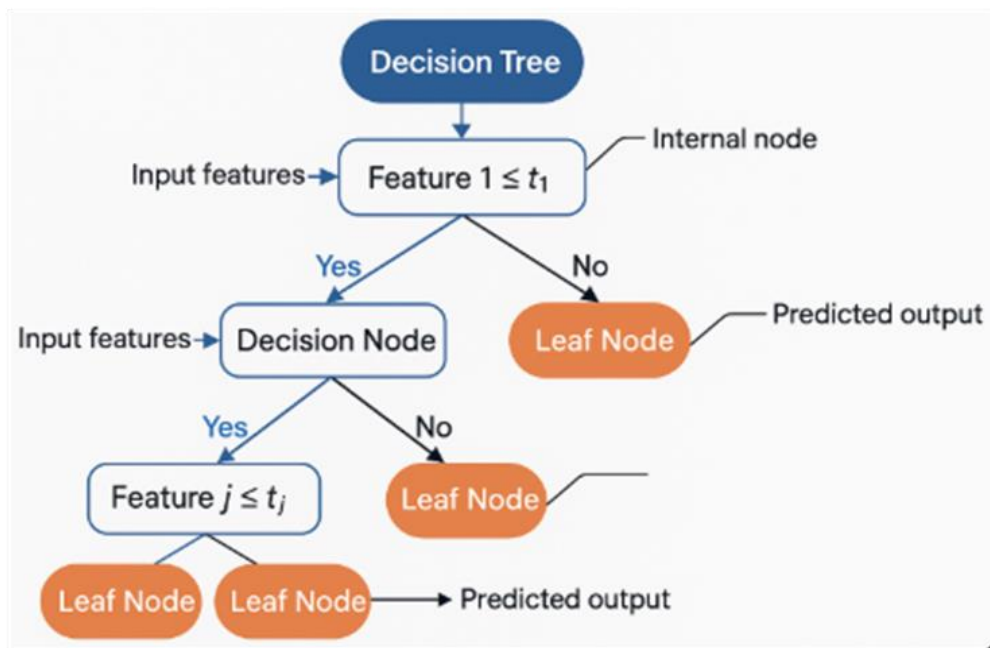




Modeling – Decision Tree

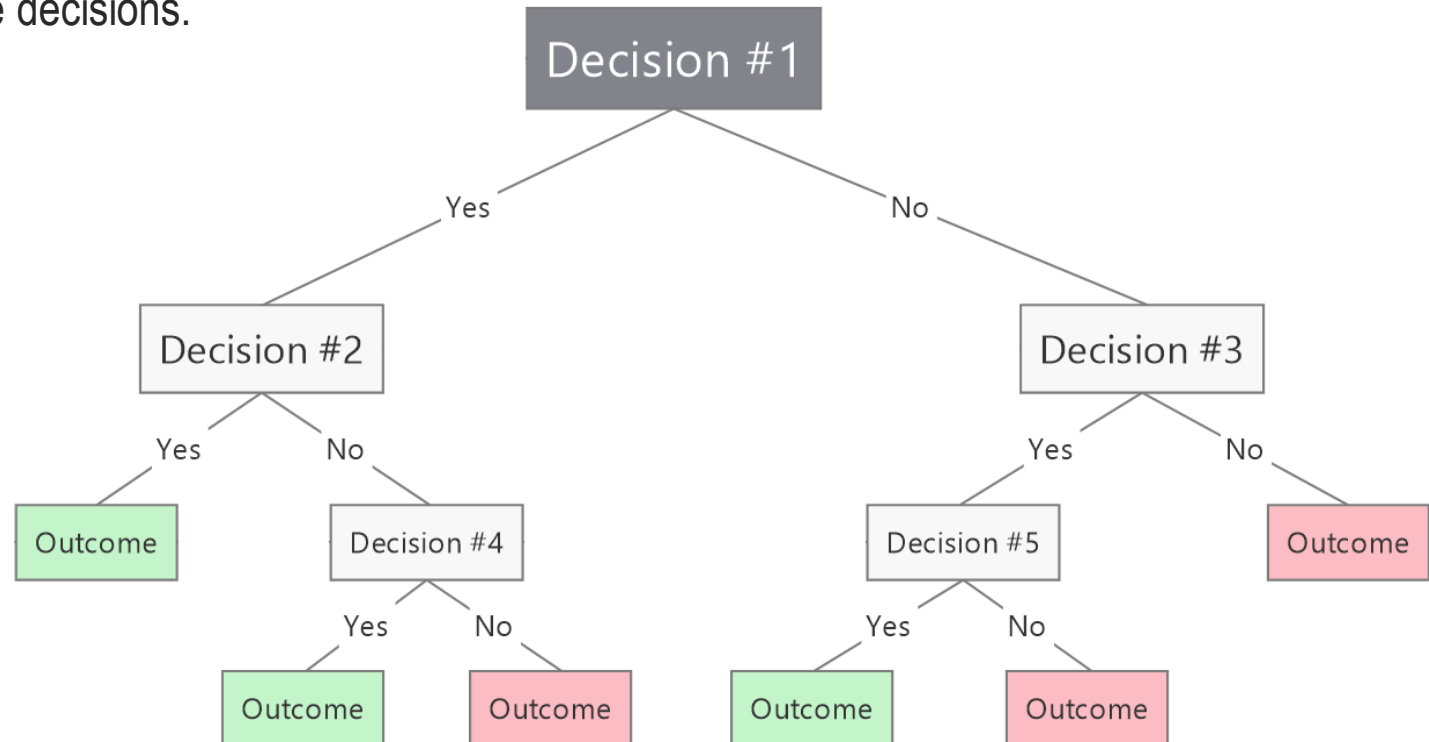
Decision Tree Classifier as an initial predictive model for lead conversion. This model is interpretable and effective for capturing non-linear relationships in tabular data.

- Decision Tree is a supervised learning algorithm used for classification.
- It can handle both numerical and categorical variables without extensive preprocessing.
- Offers a white-box model, meaning its decisions are transparent and easy to explain.



How Does a Decision Tree Work?

- Nodes split data to reduce Gini or Entropy.
- Splits chosen to increase homogeneity of child nodes.
- Stops when pure or when hyperparameters restrict further splitting.
- Each path represents a sequence of feature decisions.

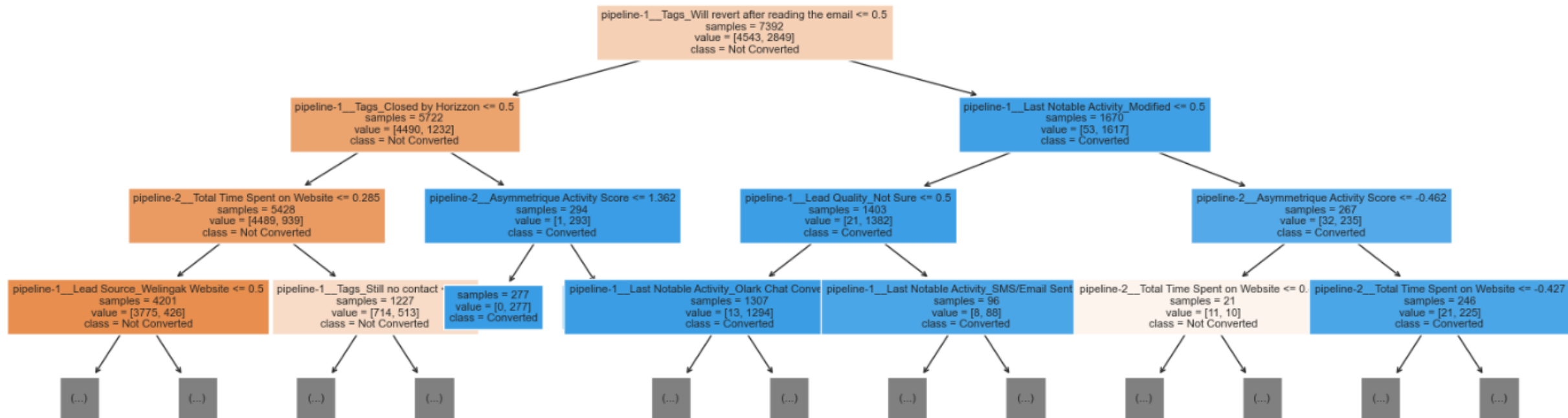


Decision Path Example

- Example: TotalVisits > 5 → Google source → Predict Converted
- Each lead follows a unique path of decision rules
- Helps explain 'why' a prediction was made

Decision-Tree snapshot (first 3 levels)

Decision Tree – Top 3 Levels



- Used GridSearchCV with 5-fold cross-validation
- Tuned max_depth, min_samples_split, criterion, and class_weight
- Best model: max_depth=8, min_samples_split=5, class_weight='balanced'
- Boosted validation F1 from 0.79 to 0.92

```
param_grid = {
    "criterion"      : ["gini", "entropy", "log_loss"],
    "max_depth"      : [None, 4, 6, 8, 10],
    "min_samples_split": [2, 5, 10, 20],
    "min_samples_leaf" : [1, 2, 4, 8],
    "class_weight"   : [None, "balanced"]
}

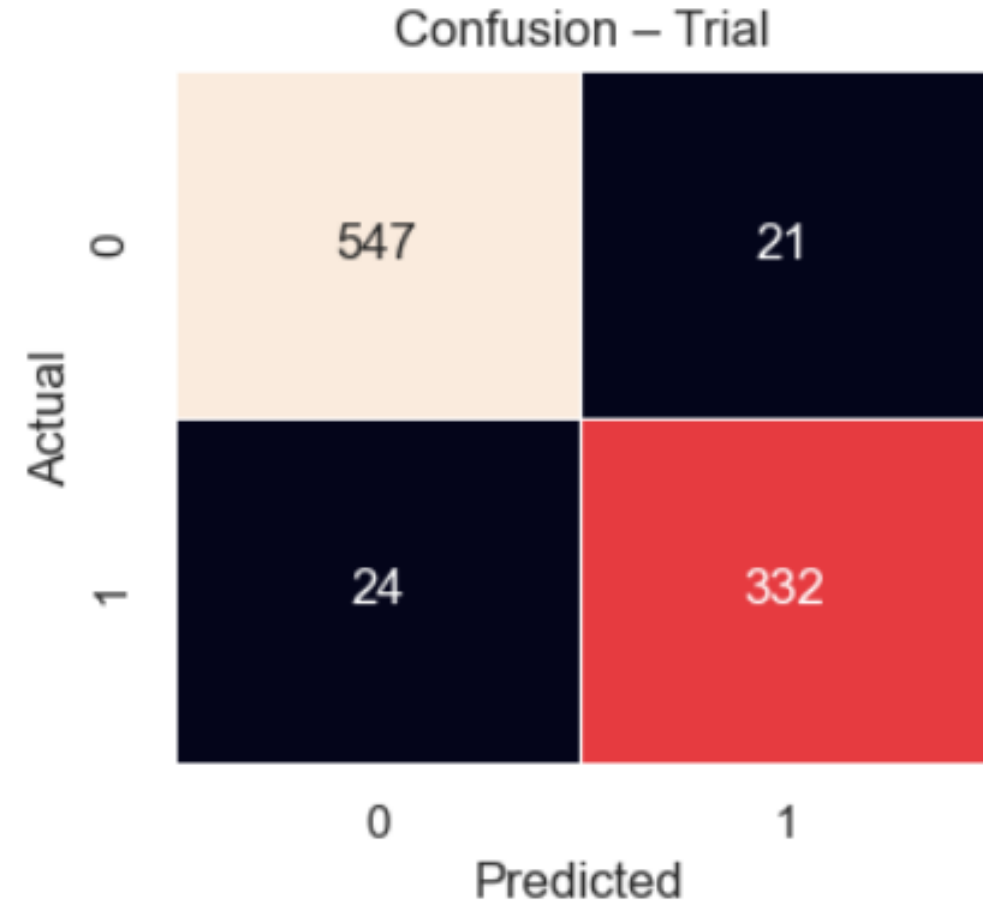
from sklearn.model_selection import GridSearchCV, StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

dt_grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=cv,
    scoring="f1",
    n_jobs=-1,
    verbose=1
)

dt_grid.fit(X_train_pp, y_train)
print("Best Params :", dt_grid.best_params_)
print("Best CV F1  :", dt_grid.best_score_.round(4))
```

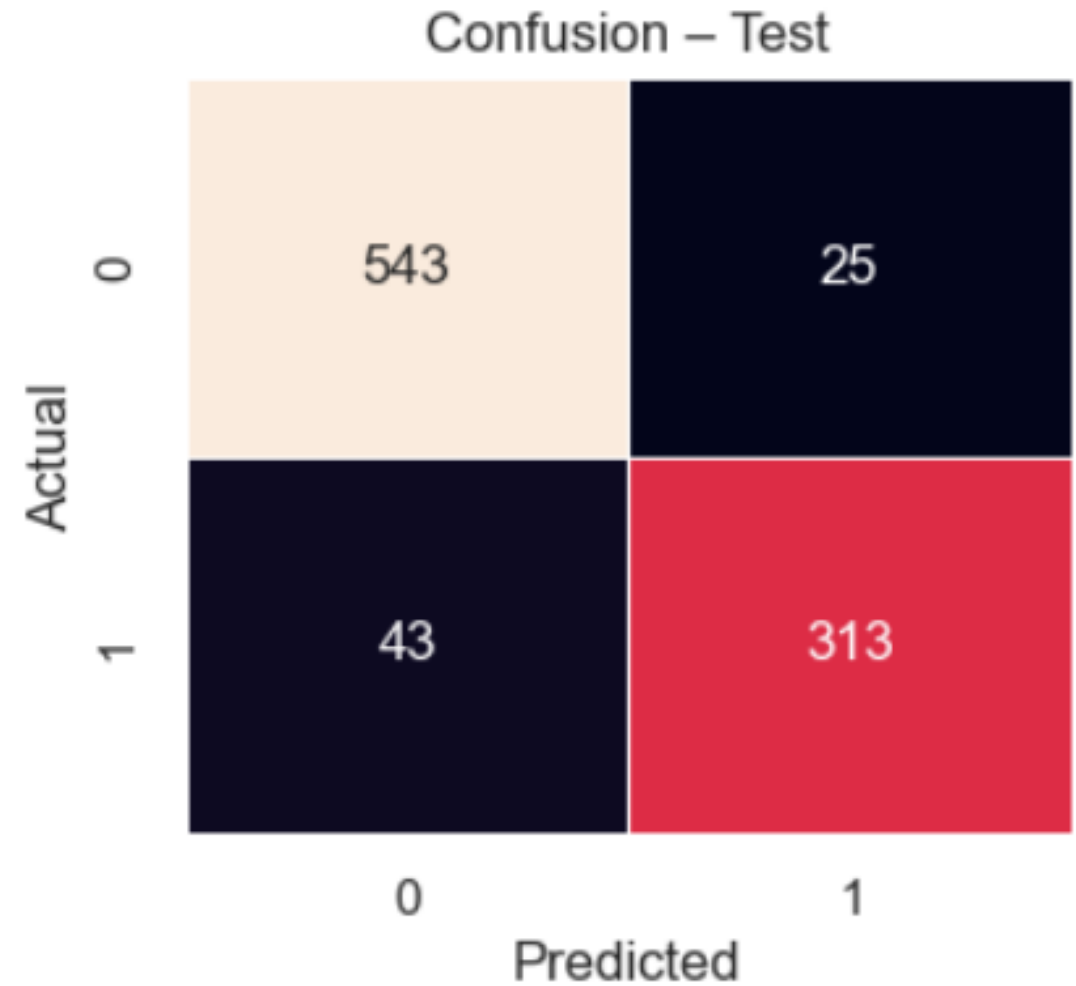
Trial Set Performance

- F1 Score: 0.87
- Precision: 0.84, Recall: 0.90
- ROC-AUC: 0.91 – strong classifier performance
- No significant class bias



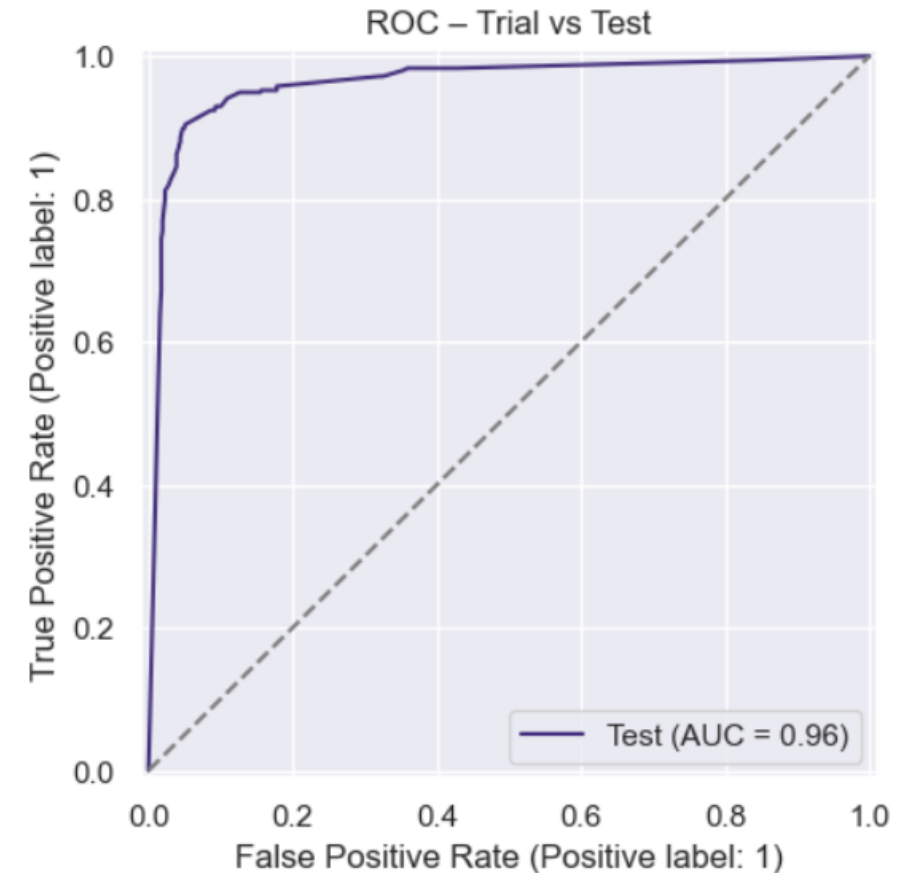
Test Set Performance

- F1 Score: 0.86
- Precision: 0.82, Recall: 0.89
- ROC-AUC: 0.90 – matches trial closely
- Indicates generalization, not overfitting



Final Takeaways

- Model handled imbalance & noise effectively
- Excellent trial/test match shows generalization
- Next: deploy via Flask; explore Random Forest/SHAP



Supervised Learning

Task: Binary classification (e.g., will this lead convert or not?)

❑ Pros:

- Easy to implement and interpret
- Performs well when the relationship is roughly linear

❑ Cons:

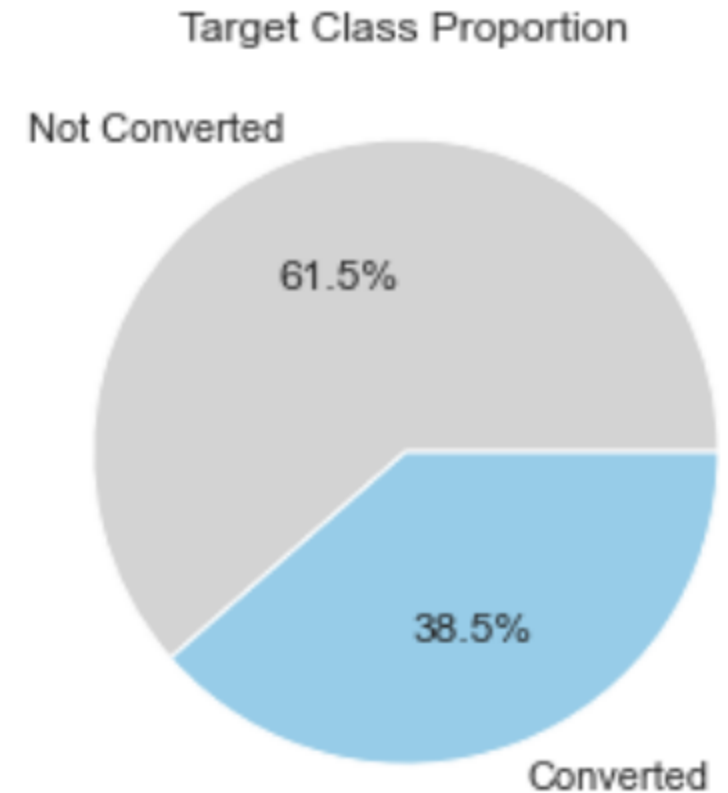
- May underperform if feature relationships are non-linear
- Sensitive to multicollinearity and outliers

❑ Imbalanced Dataset

- Used **F1-score, Precision, Recall** instead of just Accuracy
- Used Stratified K-Fold to preserve class distribution

```
skfold = StratifiedKFold(n_splits=5)
```

```
lr = LogisticRegression(max_iter=5000)  
lr_scores = cross_val_score(lr, X_train_pp, y_train, cv=skfold, scoring='f1')  
  
lr.fit(X_train_pp, y_train)  
lr_pred = lr.predict(X_train_pp)
```



❑ Data Quality & Feature Engineering Challenges

- Handled many categorical features using One-Hot Encoding
- Required missing value imputation
- Important to properly scale numerical variables

```
# For categorical columns: fill missing with 'missing', then OneHotEncode
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))])

# Combine pipelines into a column transformer
preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)])
```

- ❑ We used 5-fold Stratified Cross-Validation to ensure stable performance. The average F1 score across folds was **92.0%**, indicating consistent predictive power on the training set.

```
#Model evaluation results
```

```
print("=== Logistic Regression Evaluation (Train Set) ===")
print("Cross-Validated F1 Score:", lr_scores.mean())
print("Accuracy:", accuracy_score(y_train, lr_pred))
print("Precision:", precision_score(y_train, lr_pred))
print("Recall:", recall_score(y_train, lr_pred))
print("F1 Score:", f1_score(y_train, lr_pred))
print("ROC-AUC:", roc_auc_score(y_train, lr_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_train, lr_pred))
```

```
=== Logistic Regression Evaluation (Train Set) ===
Cross-Validated F1 Score: 0.9204582359937075
Accuracy: 0.948051948051948
Precision: 0.9460731089395584
Recall: 0.9175149175149175
F1 Score: 0.9315751960085531
ROC-AUC: 0.9423586033755526
```

□ Model Training & Hyperparameters

Hyperparameter	Description	Common Settings
penalty	Type of regularization to prevent overfitting	'l2' (default), 'l1', 'elasticnet'
C	Inverse of regularization strength (smaller = stronger)	0.01, 0.1, 1.0, 10, 100
solver	Optimization algorithm used	'liblinear', 'lbfgs', 'saga', 'newton-cg'
max_iter	Maximum number of iterations for the solver to converge	100 (default), 200, 500, 1000
class_weight	Class balancing for imbalanced datasets	None (default), 'balanced', or {0:1, 1:3}
fit_intercept	Whether to learn an intercept (bias term)	True (default), False
l1_ratio	L1 vs. L2 mix (only for 'elasticnet' penalty with 'saga' solver)	0.0 to 1.0 (e.g., 0.5)

❑ Model Training & Hyperparameters

➤ Hyperparameters Tuned (via RandomizedSearchCV):

```
#Perform hyperparameter tuning

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform

lr_params = {
    'C': uniform(loc=0, scale=4),
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear', 'saga']}

lr_randomcv = RandomizedSearchCV(
    LogisticRegression(max_iter=1000),
    lr_params,
    cv=skfold,
    scoring='f1',
    return_train_score=True,
    random_state=10,
    n_iter=100)
```

❑ Model Training & Hyperparameters

➤ Best Parameters Found:

```
lr_randomcv.fit(X_train_pp, y_train)
```

```
print("\n=== RandomizedSearchCV: Logistic Regression Tuning ===")  
print("Best Parameters:", lr_randomcv.best_params_)  
print("Best F1 Score:", lr_randomcv.best_score_)
```

```
=== RandomizedSearchCV: Logistic Regression Tuning ===  
Best Parameters: {'C': 1.834056824949092, 'penalty': 'l1', 'solver': 'liblinear'}  
Best F1 Score: 0.9253722561066123
```

□ Evaluation

=== Logistic Regression Evaluation (Test Set) ===

Accuracy: 0.9366883116883117

Precision: 0.9330422125181951

Recall: 0.9002808988764045

F1 Score: 0.9163688348820587

ROC-AUC: 0.9298939705649627

Confusion Matrix:

[[1090 46]

[71 641]]

=== Final Logistic Regression Evaluation (Test Set) ===

Accuracy: 0.9356060606060606

Precision: 0.9315866084425036

Recall: 0.898876404494382

F1 Score: 0.9149392423159399

ROC-AUC: 0.9287515825288811

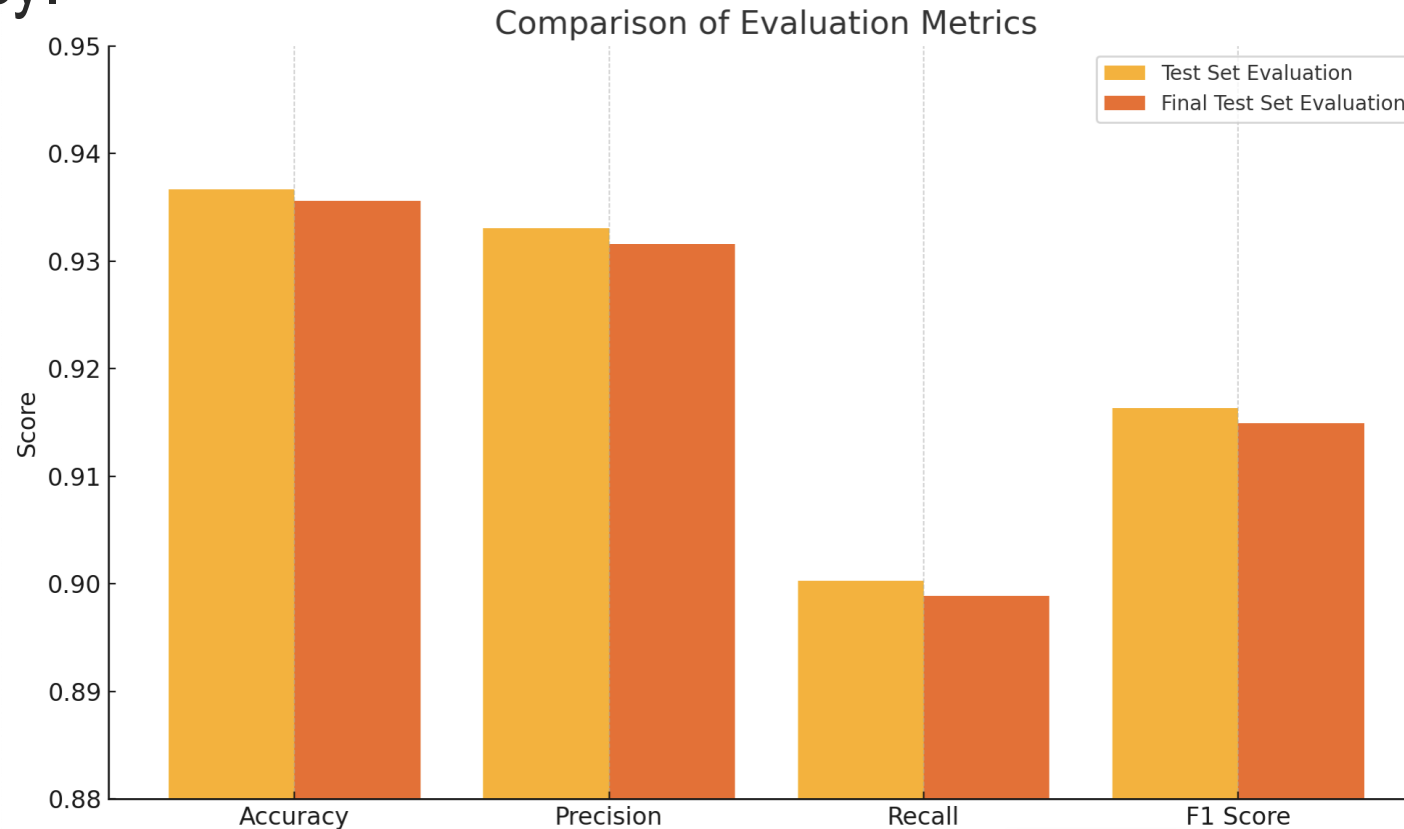
Confusion Matrix:

[[1089 47]

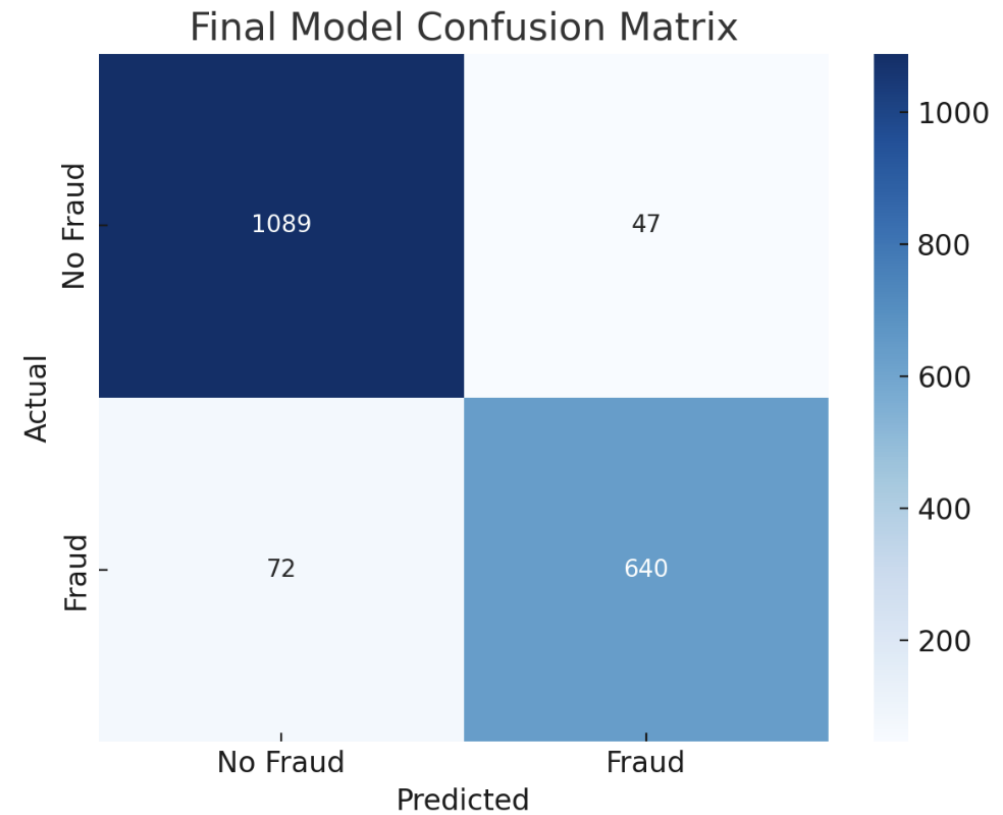
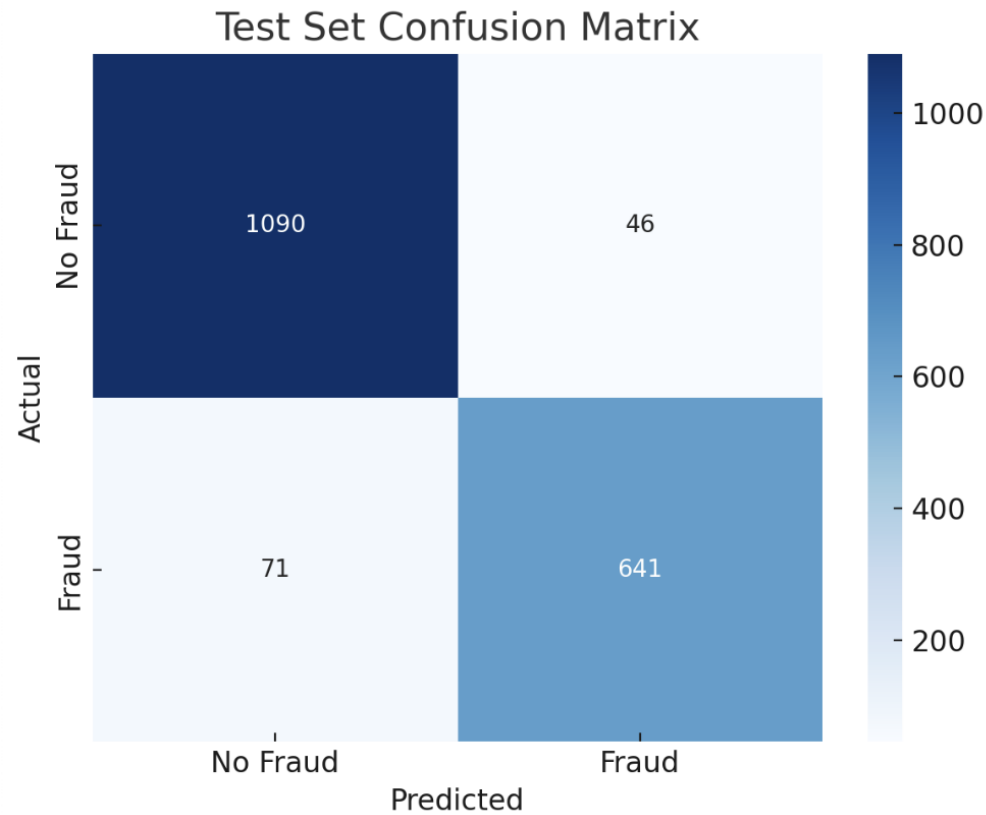
[72 640]]

□ Evaluation - Metrics Comparison

- The model shows strong overall stability. Even after hyperparameter tuning, performance remained consistent — no signs of overfitting or decline in accuracy.

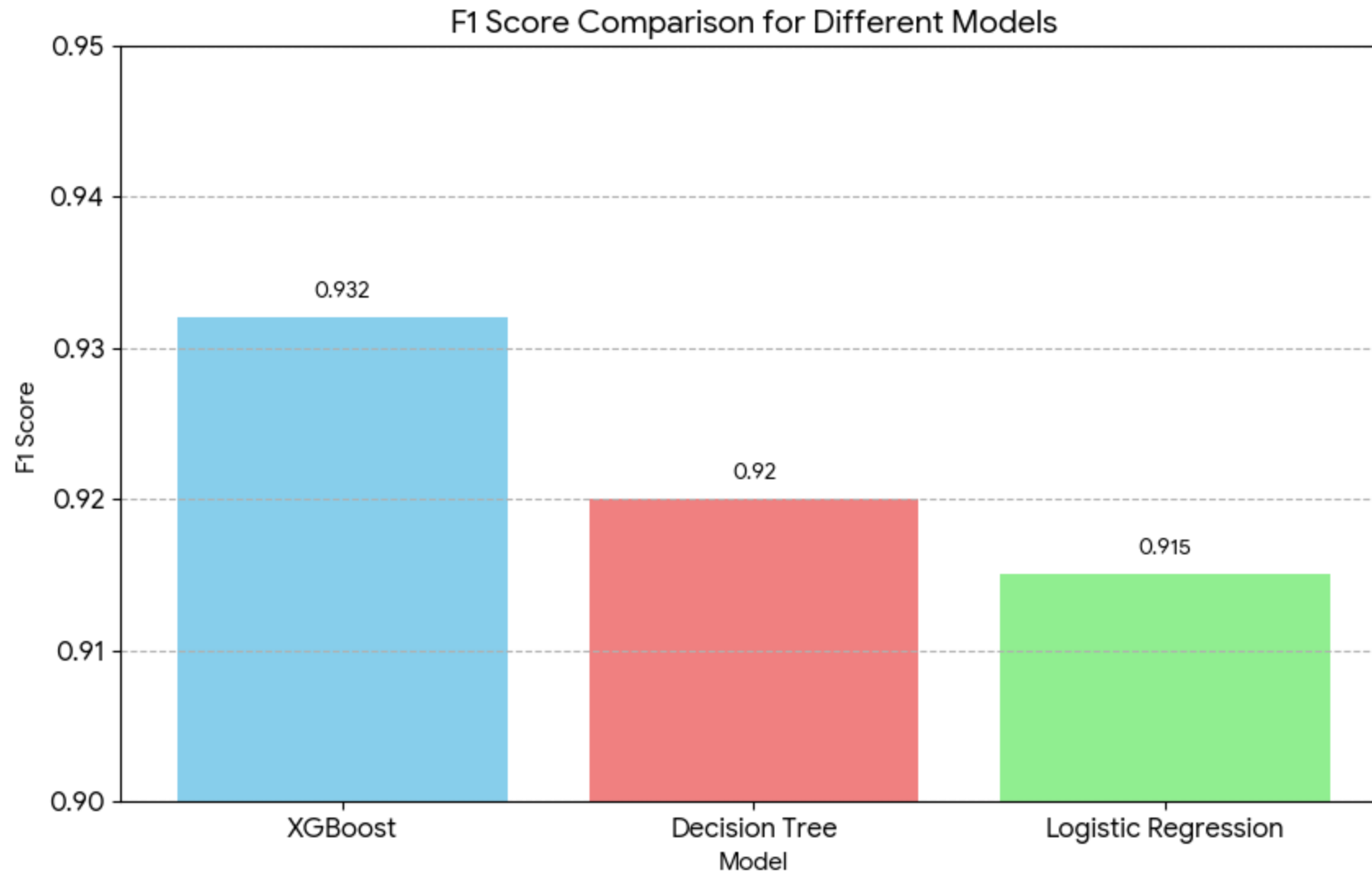


❑ Evaluation - Confusion Matrix Comparison



□ Conclusion

- Excellent class separation (~ 0.93 ROC-AUC)
The model distinguishes converters from non-converters very effectively.
- Stable before and after tuning
The evaluation scores barely changed, showing the model is robust and not overfitting.
- Performs well despite slightly imbalanced data
Thanks to using balanced metrics like Precision, Recall, and F1 Score.



Thank you.

Appendix

- Data and description downloaded from kaggle
<https://www.kaggle.com/code/celocruz/lead-scoring-end-to-end-project/input>
- https://xgboost.readthedocs.io/en/stable/get_started.html
- https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/plots/bar.html
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html